DOI - 10.32743/UniTech.2024.123.6.17714

DEVELOPMENT OF MICROSERVICES USING NESTJS: ARCHITECTURE AND PRACTICAL EXAMPLES

Viktoria Dolzhenko
Team Leader at Itez
Kaluga, Russia
E-mail: waksund@gmail.com

РАЗРАБОТКА МИКРОСЕРВИСОВ С ИСПОЛЬЗОВАНИЕМ NESTJS: АРХИТЕКТУРА И ПРАКТИЧЕСКИЕ ПРИМЕРЫ

Долженко Виктория Анатольевна
Team lead
в блокчейн-проекте Itez,
РФ, г. Калуга

ABSTRACT

This paper discusses the NestJS framework for the development of microservices. NestJS, based on Node.js and TypeScript, provides effective tools for creating scalable server applications using modern programming techniques. A feature of NestJS is an advanced dependency injection system and support for various programming patterns, which contributes to the creation of modular and testable applications. The paper analyzes in detail the key aspects of microservice architecture, including their independent deployment, scalability, and lightweight maintenance. NestJS in the context of microservice architecture provides improved complexity management and increased flexibility in the development and support of complex systems. The work highlights the advantages of using modern development tools and demonstrates practical examples of implementing microservices using NestJS.

АННОТАЦИЯ

В данной работе рассматривается использование фреймворка NestJS для разработки микросервисов. NestJS, базирующийся на Node.js и TypeScript, предоставляет эффективные инструменты для создания масштабируемых серверных приложений с помощью современных методик программирования. Особенностью NestJS является продвинутая система внедрения зависимостей и поддержка различных паттернов программирования, что способствует созданию модульных и тестируемых приложений. В работе детально анализируются ключевые аспекты архитектуры микросервисов, включая их независимое развёртывание, масштабируемость и облегчённое обслуживание. Применение NestJS в контексте микросервисной архитектуры обеспечивает улучшенное управление сложностью и повышенную гибкость в разработке и поддержке сложных систем. Работа подчёркивает преимущества использования современных инструментов разработки и демонстрирует практические примеры реализации микросервисов с использованием NestJS.

Keywords: microservices, programming, architecture, IT, software, modern technologies, program.

Ключевые слова: микросервисы, программирование, архитектура, IT, ПО, современные технологии, программа.

Introduction

NestJS is a framework for building server-side applications on Node JS. It uses progressive JavaScript and is designed with TypeScript support. Since NestJS uses the Express framework, the MVC (Model-View-Controller) pattern implementation techniques applicable to Express are also applicable to NestJS [1].

Each service is responsible for specific business capabilities and interacts with other services through well-defined APIs. This approach promotes modularity, scalability, and fault tolerance, allowing teams to develop and deploy services independently [2].

One of the notable features of NestJS is its advanced Dependency Injection (DI) system. The DI system in NestJS provides a built-in container that automatically manages dependencies in classes. This promotes modularity, testability, and code reusability, greatly simplifying complex microservice frameworks' development and maintenance.

NestJS also makes heavy use of decorators, a feature of the TypeScript language that allows metadata to be attached to classes, methods, and properties. This simplifies routing, middleware integration, input validation, and more, reducing boilerplate code and promoting cleaner and more readable code bases.

Библиографическое описание: Dolzhenko V. DEVELOPMENT OF MICROSERVICES USING NESTJS: ARCHITECTURE AND PRACTICAL EXAMPLES // Universum: технические науки : электрон. научн. журн. 2024. 6(123). URL: *https://7universum.com/ru/tech/archive/item/17714*

Support for middleware in NestJS gives developers the ability to build additional logic into the request/response loop, providing granular control over data flow and simplifying the integration of authentication, logging, and error-handling functions.

The modular NestJS architecture promotes clear code separation and organization, allowing related functionality to be encapsulated in separate modules. These modules can be easily reused and extended across different microservices, improving code modularity and making it easier to maintain.

In addition, NestJS provides support for a variety of transport protocols including HTTP, WebSockets, and TCP, allowing developers to choose the optimal communication mechanism for their microservice architectures. Flexibility in communication protocols ensures efficient integration with other services and systems, facilitating seamless data exchange and collaboration between components [4].

The main concepts that developers constantly encounter when working with NestJS include modules, controllers, and services. These elements are key to understanding and effective platform usage, providing powerful tools for realizing complex server-side applications.

1. Elements

Modules use logic into reusable code pieces (components).

```
// app.module.ts

@Module({
  imports: [],        // Other modules
  controllers: [],    // REST controllers
  providers: [],      // Services, Pipes, Guards, etc
})

export class AppModule {}
```

Figure 1. An example of writing module code [3]

Controllers are used to handle REST operations (HTTP methods).

```
// app.controller.ts

@Controller()      // Decorator indicating that the following TypeScript class is a REST controller
export class AppController {
  constructor(private readonly appService: AppService) {}    // Service available through Dependency Injection

  @Get()     // HTTP method handler
  getHello(): string {
    return this.appService.getHello();     // Calling a service method
  }
}
```

Figure 2. An example of using controllers [3]

Services are used to manage logic and functionality. Service methods are called from the controller.

```
// app.service.ts

@Injectable()       // Decorator that marks a TypeScript class a provider (service)
export class AppService {
  constructor() {}      // Other services, repositories, CQRS handlers can be accessed through Dependency Injection

  getHello(): string {
    return 'Hello World!';      // Plain functionality
  }
}
```

Figure 3. An example of services [3]

In the context of software development, certain difficulties can arise during code updates, when the entire system involved in testing and deployment must be rerun, which slows down the process considerably. In addition, in case of errors, the entire server may become unavailable, given the difficulty of localizing problems in the system repository. Additional challenges are related to expanding service functionality and introducing new features, which may require significant system refactoring.

In response to these challenges, the microservice architecture offers a solution to refactor massive monolithic applications into a more flexible and manageable form. This architectural concept aims to break down complex systems into separate, independently functioning services.

Microservices are not tied to a specific technology but are a set of architectural approaches where each service operates autonomously. The main characteristics of a microservice architecture include:

- Functional module independence, eliminating code interdependencies between services, and improving their extensibility.
- The ability to deploy each module independently speeds up application deployment and facilitates error detection.

Designing microservices should consider the potential benefits, such as simplified management and increased scalability, but also the potential increase in complexity and cost. It is important to apply microservices where it is truly warranted, based on project specifics and business requirements. Microservices usage scenarios are suitable for large projects or extensive applications [6].

In the context of the NestJS platform, the microservice architecture supports various communication protocols such as TCP that facilitate communication between individual services. This functionality includes different communication modes such as request-response for messages and event-driven mode, which allows services to efficiently exchange data and events without delay.

To create an effective microservice architecture, start by developing a simple framework. As an example, consider a Nest-based microservice platform that uses TCP as the default communication protocol. It is important to note that the main project and microservices in such an architecture reside in different directories and require separate startup. At this point, the architectural scheme is as follows: the project is divided into several microservices, each of which is placed in its directory. For example, the main service can be placed in the main-service directory, and additional microservices can be placed in the auth-service, data-service, notification-service directories etc.

Each microservice includes an app.module.ts file that defines modules, services, and controllers specific to that microservice. Interaction between microservices is performed via TCP using @MessagePattern and @EventPattern decorators for processing requests and events in asynchronous mode.

Example code for the main service that starts the TCP server:

```
import { NestFactory } from '@nestjs/core';
import { AppModule } from './app.module';
import { Transport } from '@nestjs/microservices';

async function bootstrap() {
  const app = await NestFactory.createMicroservice(AppModule, {
    transport: Transport.TCP
  });
  await app.listen(() => console.log('Microservice is listening'));
}

bootstrap();
```

Figure 4. Example code for the main service

Each microservice is launched similarly but with its own set of modules and configurations. This provides architecture flexibility and scalability, allowing each service to function and update independently [5,8].

2. Impact of microservice architecture

Microservice architecture has had a significant impact on design and modern software development approaches. This architecture is a scalable and adaptive method of creating complex software solutions where a system is broken down into many small, independently functioning services. This paper elaborates on various aspects of microservice architecture, emphasizing its advantages in the areas of scalability, maintenance, and flexibility.

In the context of software development, where size is a critical factor, microservice architecture encourages the partitioning of monolithic applications into smaller, autonomous services. These services serve as building blocks for creating large-scale digital ecosystems, each specializing in a specific business function, opening up new opportunities for innovation and improvement.

The main advantage of microservice architecture is its scalability. Unlike monolithic applications, where scaling requires the resources of the entire application, microservices allow individual components to scale independently. This leads to more efficient resource allocation, improved performance, and reduced costs.

Another significant benefit is easier support and maintenance. In monolithic systems, any changes or fixes can have unintended consequences throughout the application. Microservices, on the other hand, are isolated and have their code bases, making them easier to test and modify. This reduces the risk of bugs and side effects and makes it easier to integrate new features or technologies without interfering with the entire system.

Microservice architecture also promotes technological flexibility. With each service operating independently, teams can select the most appropriate technologies for each service according to its unique requirements. This facilitates the introduction of new technologies and allows individual services to be upgraded without risking the entire system.

However, implementing a microservice architecture entails certain complexities, such as managing interactions between services and ensuring data consistency. Fortunately, platforms such as NestJS provide robust tools for microservice management, including dependency injection, decorators, and middleware support, simplifying the development and management of microservice architectures [7].

3. Benefits of using Docker to deploy microservices

Microservice architecture offers a new level of flexibility and scalability in application development, but efficient management and deployment of multiple services require the right tools. In this context, Docker stands out, which greatly simplifies deploying microservices by ensuring that they work consistently across a variety of environments.

Docker, an open-source platform, provides capabilities to build, package, and distribute applications in the form of lightweight, portable containers.

One of the key benefits of using Docker in microservices deployments is its scalability. With Docker, individual services can be containerized and scaled independently of each other, allowing for efficient resource allocation and application stability as workloads increase.

In addition, Docker provides increased isolation and security for microservices. Each container operates in a separate, isolated environment, which minimizes the risks associated with errors or failures in one container affecting the operation of others. This isolation reduces the likelihood of cascading failures and improves overall system stability. Docker's approach to containerization also adds a layer of security because containers are isolated from the underlying host operating system, reducing the risks of unauthorized access and data leaks.

Managing a complex microservice architecture can be a daunting task, but Docker facilitates this process through Docker Compose, a tool that defines and organizes multi-container applications with a simple YAML file. It can declaratively set service parameters, dependencies, and network settings, simplifying deployment and scaling with a single command.

An additional benefit is access to a wide ecosystem of pre-configured images on Docker Hub, the official Docker image registry. These images can serve as the foundation for building microservices, saving development time, and facilitating collaboration between teams through the ability to share and distribute images [9].

Conclusion

In summary, microservice development using NestJS demonstrates significant advantages in modern application development. The modular architecture and dependency enforcement system of the framework contribute to the ease of integration and testing of individual services. This framework allows applications to scale efficiently and ensures reliability through component independence. NestJS improves development and support processes by providing developers with flexible tools for solving specific tasks. Support for various transport protocols and advanced routing capabilities make NestJS an ideal choice for implementing microservice architecture in projects of varying complexity. As a result, NestJS proves its effectiveness in creating powerful, flexible, and scalable server solutions.

References:

1. Building Microservices using NestJS and KafkaJs. [Electronic resource] – Access mode: https://gaiyaobed.hashnode.dev/building-microservices-using-NestJS-and-kafkajs .– (accessed 04/27/2024).
2. Building Microservices with JavaScript and Node.js. [Electronic resource] – Access mode: https://www.tutorialspoint.com/building-microservices-with-javascript-and-node-js .– (accessed 04/27/2024).

3. Building Microservices with NestJS is that simple! [Electronic resource] – Access mode: https://dev.to/vue-storefront/building-microservices-with-NestJS-is-that-simple-4afm .– (accessed 04/27/2024).
4. How to create a NestJS Microservice using the Request Response approach? [Electronic resource] – Access mode: https://progressivecoder.com/how-to-create-a-NestJS-microservice-using-request-response-approach / .– (accessed 04/27/2024).
5. NestJS for growing development: why is it so difficult and why is it still so? [Electronic resource] – Access mode: https://habr.com/ru/companies/sravni/articles/704594 / .– (accessed 04/27/2024).
6. NestJS microservices architecture practices. [Electronic resource] – Access mode: https://www.mo4tech.com/NestJS-microservices-architecture-practices.html .– (accessed 04/27/2024).
7. The Power of Microservices Architecture. [Electronic resource] – Access mode: https://www.devban.com/scalable-applications-NestJS-microservices-docker/#The_Power_of_Microservices_Architecture .– (accessed 04/27/2024).
8. This repository shows how you can build an API with microservice architecture using NestJS. [Electronic resource] – Access mode: https://github.com/Denrox/NestJS-microservices-example .– (accessed 04/27/2024).
9. Writing Microservices using NestJS. [Electronic resource] – Access mode: https://itsmebhavin.wordpress.com/2020/01/28/writing-microservices-using-NestJS / .– (accessed 04/27/2024).